

Deep Learning in Asset Pricing

Chen, Pelger & Zhu (Management Science, 2024): replicating the adversarial SDF in PyTorch, what broke along the way, and what we built on top of it.

Aditya Aryan · Giancarlo Alves · Hashim Almodamagha · Thomas Claudel · Alex Roesler

September 11, 2026

Where we left off, and what is new today

LAST TERM: A REFEREE REPORT

The claim, and our two objections

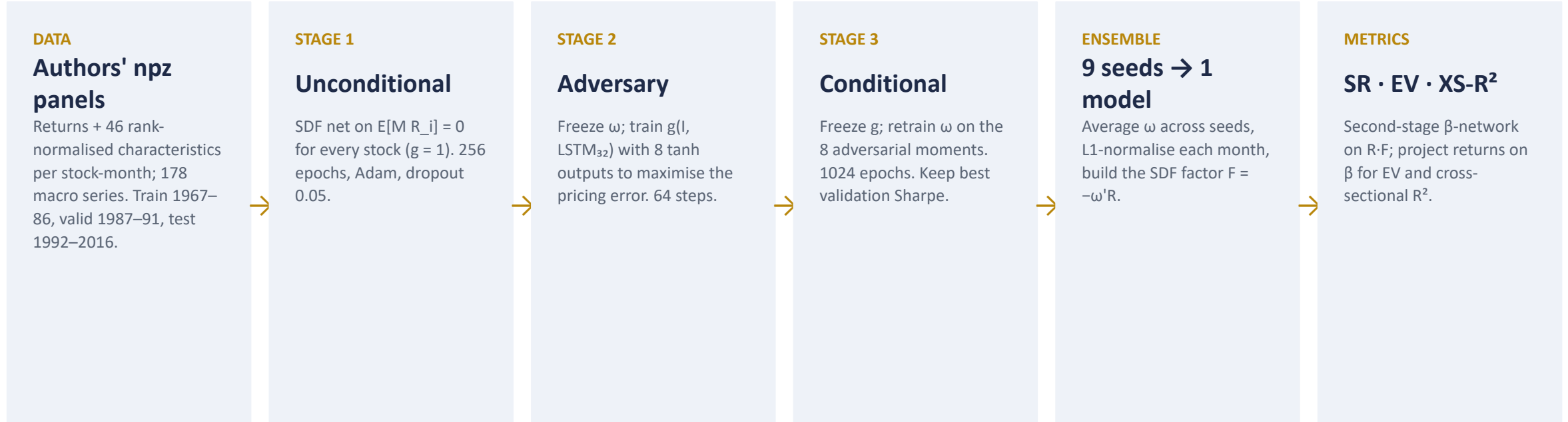
- The paper estimates the SDF for ~10,000 U.S. stocks with three networks: an LSTM for macro states, a feedforward net for the SDF weights ω , and an adversary that picks the hardest test assets. The loss is the no-arbitrage condition $E[M R] = 0$, not a forecast error.
- Headline: out-of-sample Sharpe 2.6 (annual), EV 8%, XS- R^2 23%; beats FFN, elastic net and OLS.
- Our objection 1: fitted once in 1986, never re-estimated, reported as one number over 25 years.
- Our objection 2: turns over $\approx 100\%$ of the book every month, every Sharpe ratio is gross of costs.

TODAY

From reading the paper to running it

- Replicated the paper end-to-end: simulation (Table II) and U.S. equities (Table III) with the authors' own data.
- Ported the (dead) TensorFlow-1.12 code to PyTorch; every design choice is now inspectable and switchable.
- Ran the ablations that isolate why it works: no adversary, no macro, raw macro without LSTM.
- Built both 230P improvements into the code: transaction costs inside the objective, and a year-by-year test.
- Added a third extension the paper skips: enforcing an admissible (positive) SDF.

What we replicated, and how the pieces fit



REPLICATED

Table II (simulation, two DGPs) · Table III (U.S. equities: SR, EV, XS-R² for GAN vs. benchmarks) · Figure 6 ablations (no adversary, no macro, 178 raw macro series) · Figures 8, 9, 13 (variable importance, β -sorted deciles, LSTM macro states). Same architecture as the paper's Table I optimum: FFN 2×64, 4 SDF states, 32 adversary states, 8 moments, lr 0.001.

The data: what the authors share, and what they don't

Split	Months	Window	Permno in split	Stock-months	Avg stocks / month
Train	240	1967–1986	3,686	336,113	1,400.5
Valid	60	1987–1991	3,347	132,167	2,202.8
Test	300	1992–2016	7,141	750,275	2,500.9

1,218,555 stock-months. Shapes and windows match the paper exactly.

Inputs, exactly as in the paper

- 46 characteristics (past returns, investment, profitability, intangibles, value, frictions), each ranked cross-sectionally every month into $[-0.5, 0.5]$. A stock enters only when all 46 are available: $\sim 31,000$ CRSP names $\rightarrow \sim 10,000$.
- 178 macro series: 124 FRED-MD, the 46 characteristic medians, 8 Welch–Goyal predictors; McCracken–Ng transformations, then standardised with training-window moments only.
- Missing return = -99.99 sentinel $\rightarrow$ boolean mask; the loss weights each stock by its number of months T_i .

WHAT IS NOT SHARED

Three gaps we had to work around

- No PERMNO identifiers. The stock index is consistent inside a split but not across splits, and each split holds only 3.7k / 3.3k / 7.1k names — the paper's " $\sim 10,000$ " is the union over the whole sample. Enough for turnover, not enough to join anything new.
- No raw CRSP/Compustat: the characteristic construction (Table A.VII) cannot be re-run, so 2017–2026 cannot be appended without WRDS.
- Macro is pre-transformed and final-vintage: our 230P real-time critique still stands and cannot be tested from this data.

The objective we actually optimise

PRICING LOSS (authors' code, line for line)

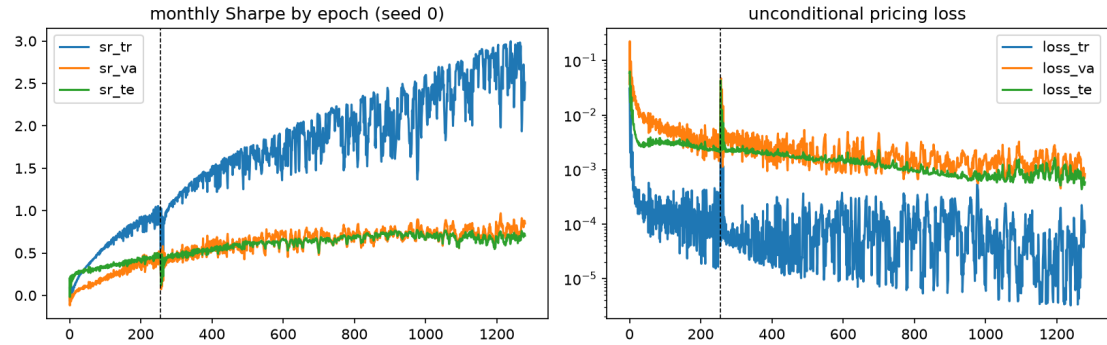
$L(\omega, g) = \text{mean over } j, i \text{ of } (T_i / T_{\text{max}}) \cdot [(1/T_i) \sum_t M_{\{t+1\}} \cdot R_{\{t+1,i\}} \cdot g_{\{j,t,i\}}]^2$

$M_{\{t+1\}} = 1 + \sum_i \omega(l_{\{t,i\}}, h_t) R_{\{t+1,i\}} \quad h_t = \text{LSTM}_4(\text{macro}_{\{1..t\}}) \quad g_{\{j\}} = \tanh(\text{FFN}(l, \text{LSTM}_{32}(\text{macro})))$

Stage 1 sets $g \equiv 1$. Stage 2 maximises L in g . Stage 3 minimises L in ω with g frozen.

SELECTION & ENSEMBLE

- Every epoch: unconditional loss and Sharpe on train / valid / test. Checkpoints kept for best validation loss (used to move between stages) and best validation Sharpe (used for the ensemble).
- Nothing in the 1992–2016 window is used for any decision; it is printed for the training-curve plot only.
- Ensemble = mean of raw ω over 9 seeds, then L1-normalised per month, so F is the return on \$1 gross.



Seed 0. At the dashed line the SDF net is reset to its best-validation-loss weights and the adversarial moments take over: training Sharpe then runs away to 3.0 while validation and test stay flat at 0.7 — the ensemble is what keeps that gap honest.

Hyperparameter	Paper (Table I)	Ours
SDF net layers × units	2 × 64	2 × 64
SDF / adversary LSTM states	4 / 32	4 / 32
Adversary layers / moments	0 / 8	0 / 8
lr · dropout · epochs	0.001 · 0.95 · 256/64/1024	same

Four things that did not just work

1 · THE CODE IS DEAD

TensorFlow 1.12, Python 3.6, 2019 APIs

placeholders, dynamic_rnn, boolean_mask, tf.train.Saver: none of it installs on a 2026 machine. Getting the GPU was its own detour — pip resolved the CPU wheel until we installed torch from the cu128 index alone.

HOW WE DEALT WITH IT

Rewrote the model in PyTorch against the TF graph, function by function: same data layer, same masked loss with the T_j weighting, same three-stage schedule and checkpoint rules. Verified on a synthetic panel with a planted SDF, then on the paper's own simulation DGP, before touching CRSP data.

2 · DEFAULTS ARE PART OF THE MODEL

The first full run had the shape but not the numbers

Test Sharpe 0.63 against the paper's 0.75, and the cost-aware extension was nearly worthless at 0.09. Nothing was obviously broken. Framework defaults were silently doing the work: initialisers, where Adam's epsilon sits, whether dropout is on for a frozen network.

HOW WE DEALT WITH IT

Line-by-line audit against their graph and notebook, then eight fixes: TF Glorot init with LSTM forget bias 1, a tf.train.Adam-exact update with a fresh optimiser per stage, dropout left active on the frozen nets, the adversary's 4×64 loop, ignore_epoch 32. Headline 0.63 → 0.68; the cost extension 0.09 → 0.51.

3 · THE SEARCH IS TOO BIG

384 configs × validation × 4 finalists × 9 seeds

Even on an H100 a full re-tune is days, not an evening. And selection on validation Sharpe means the paper's test numbers are themselves conditional on that search.

HOW WE DEALT WITH IT

Fixed the architecture at their reported optimum (Table I) and spent the compute on 9 seeds per model plus the ablations — about one fortieth of their budget. Then report the seed dispersion, which the paper does not.

4 · STATE THAT CARRIES OVER

LSTM memory across train → valid → test

The macro state at January 1992 depends on 25 years of history. Evaluate the test window from a zero state and the model prices with amnesia.

HOW WE DEALT WITH IT

Run the LSTM over the concatenated macro history and slice the window — what the authors' getNextInitialState does. The same trick lets us read the four hidden states over the whole sample.

On the paper's own DGP, the port recovers the SDF

$R = \beta F + \varepsilon$, $F \sim N(0.32, 0.1)$, $\varepsilon \sim N(0,1)$, $N = 500$, $T = 250 / 100 / 250$. Setup 1: $\beta = C_1 \cdot C_2$ (a pure interaction). Setup 2: $\beta = C \cdot \text{sign}(h_t)$, h_t a noisy sine cycle observed only through its trended increment.

Setup 1: two characteristics	SR train	SR valid	SR test	EV test	XS-R ² test
Population SDF (paper)	0.96	1.09	0.94	0.17	0.17
GAN (paper)	0.98	1.11	0.94	0.13	0.07
GAN (ours, 3 seeds)	0.99	0.92	1.00	0.17	0.12
FFN forecast (paper / ours)	0.94 / 0.97	1.04 / 0.92	0.89 / 1.02	0.05 / 0.17	-0.33 / 0.13
Linear (paper / ours)	0.07 / 0.12	-0.10 / 0.04	0.01 / -0.09	0.00 / 0.00	0.01 / 0.00

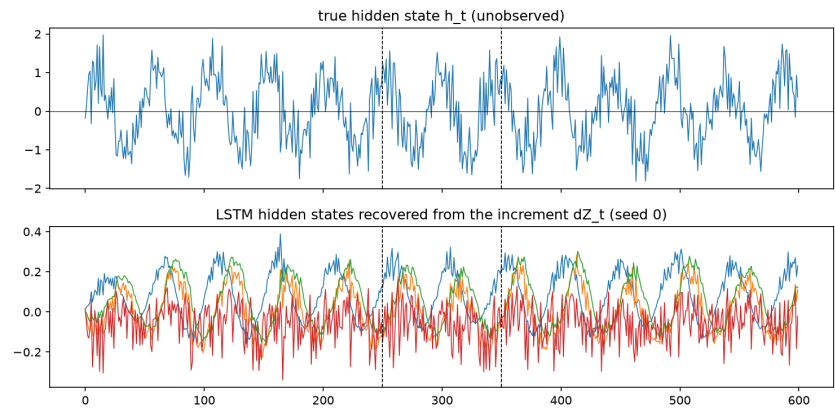
Setup 2: char + hidden macro state	SR test	EV test	XS-R ² test
Population (paper)	0.86	0.17	0.15
GAN with LSTM (paper / ours)	0.64 / 0.60	0.17 / 0.01	0.15 / 0.00
FFN, last macro obs (paper / ours)	0.06 / 0.02	0.02 / 0.16	0.02 / 0.20

Setup 2 caveat: the loading is $C \cdot \text{sign}(h_t)$, and our θ -network sees characteristics only — as the authors' own config does — so no function of C can represent it and EV / XS-R² are ≈ 0 by construction, not by failure. The SDF weights, which do see the state, still reach 0.60 against the paper's 0.64 — though our population SDF scores 0.97 there against their 0.86, so the two calibrations are not identical. Our FFN and LS benchmarks are our own code (the authors released no simulation script) and are stronger than theirs.

WHAT THIS BUYS US

- Setup 1 is the clean check on the port: GAN test Sharpe 1.00 against a population SDF of 1.02 and the paper's 0.94, while the linear model finds nothing in the interaction — exactly the paper's result.
- Costs minutes on a GPU, so any change to the objective can be regression-tested here before it touches CRSP.

SETUP 2 · THE LSTM RECOVERS A CYCLE IT NEVER SEES



The ordering replicates; the level lands 9 % low

Model	SR train	SR valid	SR test (monthly)	SR test (annual)	EV test	XS-R ² test
GAN, hidden macro states (paper)	2.68	1.43	0.75	2.60	0.08	0.23
GAN, hidden macro states (ours, 9 seeds)	2.99	1.36	0.68	2.36	0.084	0.228
spread across the 9 seeds (test SR)			0.283 – 0.750			
UNC: no adversary (paper / ours)	1.93 / 1.52	1.33 / 0.72	0.53 / 0.45	1.84 / 1.57	0.07 / 0.104	0.19 / 0.287
GAN, no macro (paper / ours)	1.90 / 1.75	1.35 / 1.49	0.69 / 0.62	2.39 / 2.15	— / 0.101	— / 0.258
GAN, 178 raw macro, no LSTM (paper / ours)	1.07 / 1.03	0.05 / 0.02	0.05 / 0.07	0.17 / 0.23	— / 0.117	— / 0.279
FFN forecast (paper)	0.45	0.42	0.44	1.52	0.04	0.15
Elastic net (paper)	1.37	1.15	0.50	1.73	0.04	0.19

THE HEADLINE: 0.68 VS 0.75

9 % low, on one fortieth of the compute and with no hyperparameter search. Train came in above the paper (2.99 vs 2.68), so our fit ran slightly further before validation stopped it. EV and XS-R² land on the paper: 0.084 / 0.228 vs 0.08 / 0.23.

THE ORDERING REPLICATES — BUT NOT ON EV

Sharpe ranks exactly as in the paper: adversary 0.68 > none 0.45, and states 0.68 > no macro 0.62 > raw macro 0.07. EV and XS-R² do not rank at all: every ablation, including the run that collapses to 0.07, scores as high or higher.

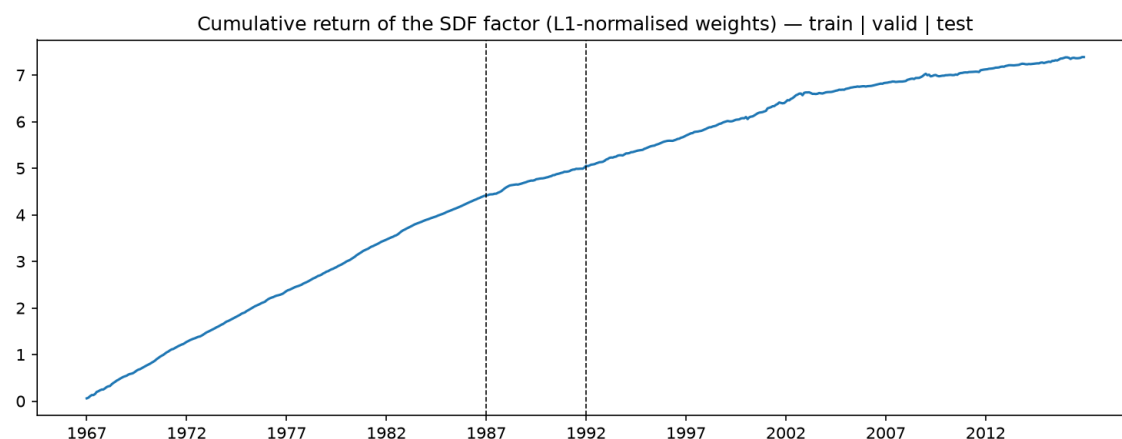
WHAT THE PAPER NEVER REPORTS

Across 9 seeds the test Sharpe runs 0.28 to 0.75 — a spread wider than the paper's gap between the GAN and OLS. The ensemble (0.68) beats the average seed (0.49) and 8 of the 9, but not seed 0.

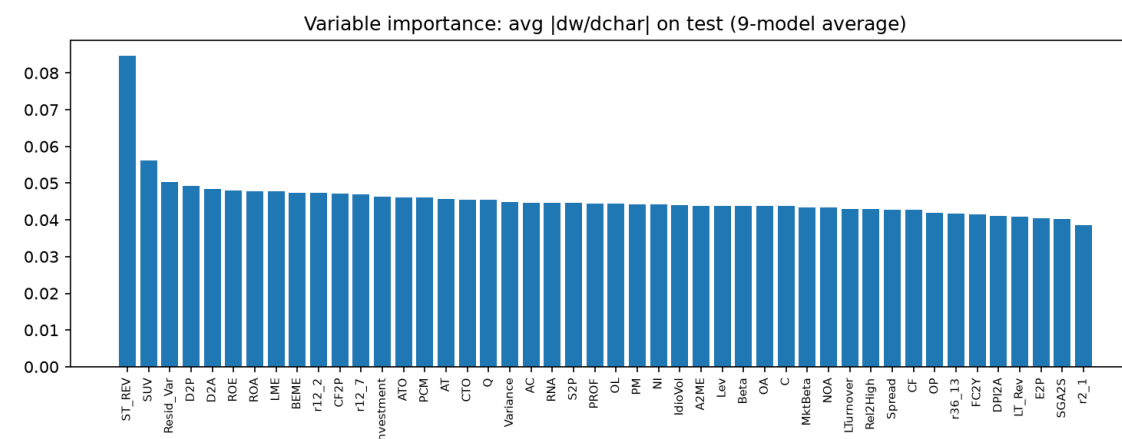
Ours = 9-seed ensembles of best-validation-Sharpe checkpoints; the raw-macro ablation has 4 seeds. XS-R² is the T_i-weighted definition the authors' notebook prints — unweighted it is 0.045.

Two of the paper's three pictures come back

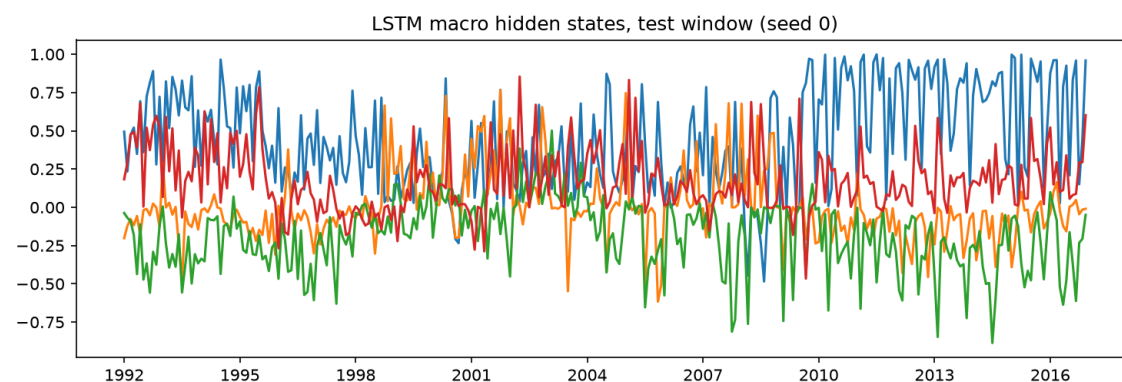
① Cumulative SDF factor — 9-seed ensemble, L1-normalised weights



② Variable importance — mean $|\partial\omega/\partial\text{char}|$ on test, 9-model average



③ The four LSTM macro states over the test window



HOW TO READ THESE

- ① Still climbing after 1992, and the drawdown is tiny: worst month -4.5% , max drawdown 5.5% over 25 years. This is the picture that makes the paper's claim look strong.
- ② The shape, not the names: short-term reversal (0.085) and SUV (0.056) stand out, then 43 of 46 characteristics sit flat between 0.050 and 0.038. The paper's Fig. 8 is far more concentrated — we do not replicate which characteristics matter.
- ③ The honest one. Paper Fig. 13 has four smooth cyclical states peaking in recessions the model never saw. Ours are high-frequency and show no visible business-cycle structure. The states are never written to disk, so we cannot quantify it — we can only report it.

The 2.36 survives 25 bps, not 50

0.96

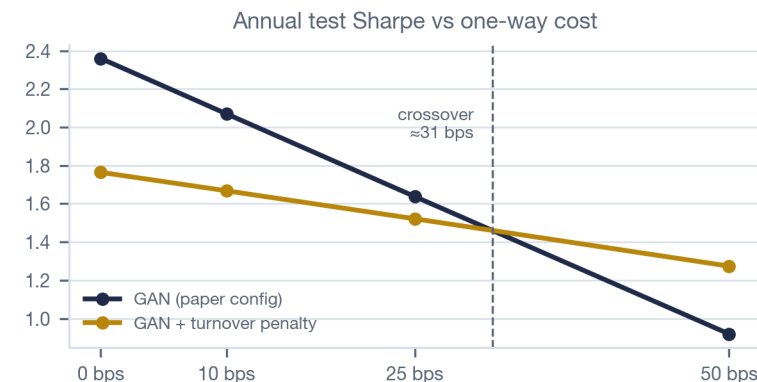
monthly turnover of the L1-normalised book on test — we guessed ≈ 1.0 in 230P

1.64

annual test Sharpe net of 25 bps one-way costs (2.36 gross)

1.52

same for the cost-aware model — at 25 bps the penalty still loses



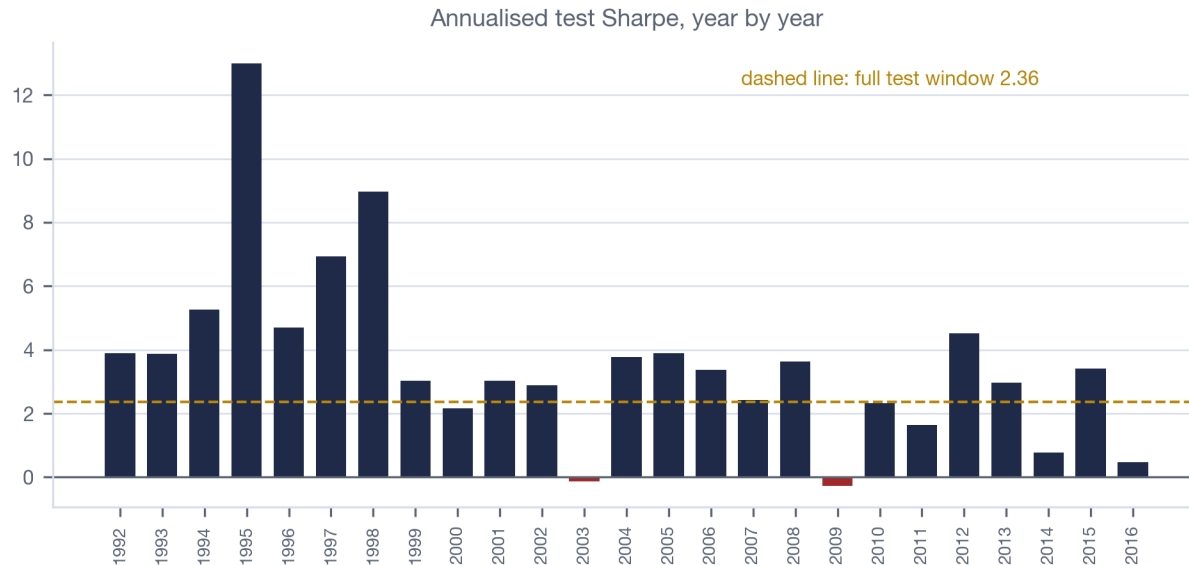
POST HOC (what we proposed in 230P)

Each month subtract cost $\times$ turnover from the SDF-factor return and recompute the Sharpe at 0 / 10 / 25 / 50 bps. The portfolio never saw the cost, so this is a floor, not an estimate. Computed for every run in results.json. The paper's headline 2.6 is the 0 bps column and nothing else.

INSIDE THE LOSS (new)

Add $\lambda \cdot E_t \sum_i |\tilde{\omega}_{\{t,i\}} - \tilde{\omega}_{\{t-1,i\}}|$ to the pricing loss on L1-normalised weights, $\lambda = 0.002$. Turnover $0.96 \rightarrow 0.24$ and gross Sharpe $2.36 \rightarrow 1.76$: the network pays four fifths of its trading to keep three quarters of its Sharpe. Net of cost it loses at 25 bps (1.52 vs 1.64) and wins at 50 (1.27 vs 0.92). Crossover ≈ 31 bps one-way.

Two things the paper reports as one number, or not at all



OUR 230P IMPROVEMENT 1, DELIVERED

First five test years average 6.15 annualised, last five 2.42; 1992–2004 averages 4.72 against 2.43 for 2005–2016, with two negative years (2003, 2009). The single number 2.60 hides a model that is three times better in its first decade than its last. The expanding-window refit needs PERMNOs and stays on the 230ZB list.

ADMISSIBLE SDF (NEW)

A pricing kernel must be positive

- No-arbitrage requires $M_{t+1} > 0$. The paper checks this in-sample and never imposes it; nothing stops the estimated M from going negative in a crash month out of sample.
- We add $\lambda \cdot E[\max(0, -M_{t+1})^2]$ to the loss (one line). $\lambda = 1$.
- Result: $M < 0$ in 0 of 300 test months for the baseline already (minimum $M = 0.944$), and 0 of 300 with the penalty (minimum 0.946). The constraint never binds.
- It is not free: test Sharpe falls $0.68 \rightarrow 0.23$ and turnover $0.96 \rightarrow 0.79$ for no measurable gain in admissibility.
- Why it is unmeasurable: we report M on L1-normalised weights, so M sits in $[0.94, 1.05]$ by construction, while the penalty acts on the raw weights during training — and only the normalised weights are saved. Fixing that is a two-line change we did not make before the deadline.

What we would do with WRDS access and another term

EXTEND THE SAMPLE

Rebuild the 46 characteristics from CRSP/Compustat, append 2017–2026, and test the 1986 model through COVID and the 2022 rate cycle. Then the expanding-window refit from 230P.

INFERENCE, AND SAVE MORE STATE

Block-bootstrap the test months across the 9-seed ensemble to put confidence bands on SR, EV and XS- R^2 differences — with a seed spread of 0.28 to 0.75, no comparison in this literature should be made without them. Also dump the raw weights and the LSTM states, the two things we could not check today.

STRESS THE ADVERSARY

Vary the number of moments (4 / 8 / 16), run more than one adversarial round, and hold out characteristics from the adversary. Cheap with the current code.

THE RL LINK (THIS COURSE)

The cost-aware SDF is a sequential decision: state = (characteristics, macro state, current book), action = ω , reward = pricing error + trading cost. A natural actor–critic formulation of the same problem.

Questions?

Code, configs and every number in this deck: github.com/Aroesler1/dlap-torch-replication — `run.py`, `simulate.py`, `summarize.py`, `deck/review.md`

Backup slide follows: every deliberate deviation from the authors' implementation.

Deviations from the authors' implementation (for the record)

Eight fidelity gaps were found by a line-by-line audit against the authors' graph and notebook, and all eight were fixed before the numbers in this deck were produced (test Sharpe 0.63 → 0.68, cost-aware extension 0.09 → 0.51). What remains below is what still differs, on purpose.

Item	Authors (TF 1.12)	Ours (PyTorch)	Effect
Hyper-parameters	384-config grid, best 4 on validation, 9 seeds each	Their Table I optimum, 9 seeds, no search	Same architecture, ~1/40 of the compute. Most likely source of 0.68 vs 0.75
Random draws	TF RNG, float32 reduction order	PyTorch RNG, float32 reduction order	Moves seed-level paths only; our seed spread is 0.283–0.750
Simulation benchmarks	No code released	Our own FFN and LS implementations	Ours are stronger than the paper's reported rows (setup 1 FFN EV 0.17 vs 0.05)
Empirical benchmarks	FFN, elastic net, OLS on CRSP	Not run — the paper's values are quoted	Slide 8 rows marked (paper)
EV / XS-R ²	β network, T_i-weighted (notebook prints WXS _R 2)	Same, plus the unweighted value and an ω-projection	Extra diagnostic; weighted 0.228 vs unweighted 0.045
Extensions	—	turnover_penalty, sdf_hinge, yearly Sharpe, net-of-cost Sharpe	Opt-in flags, both off in configs/gan.json; the replication is pure